



Operating Systems

[1500WETOPS]

José Oramas



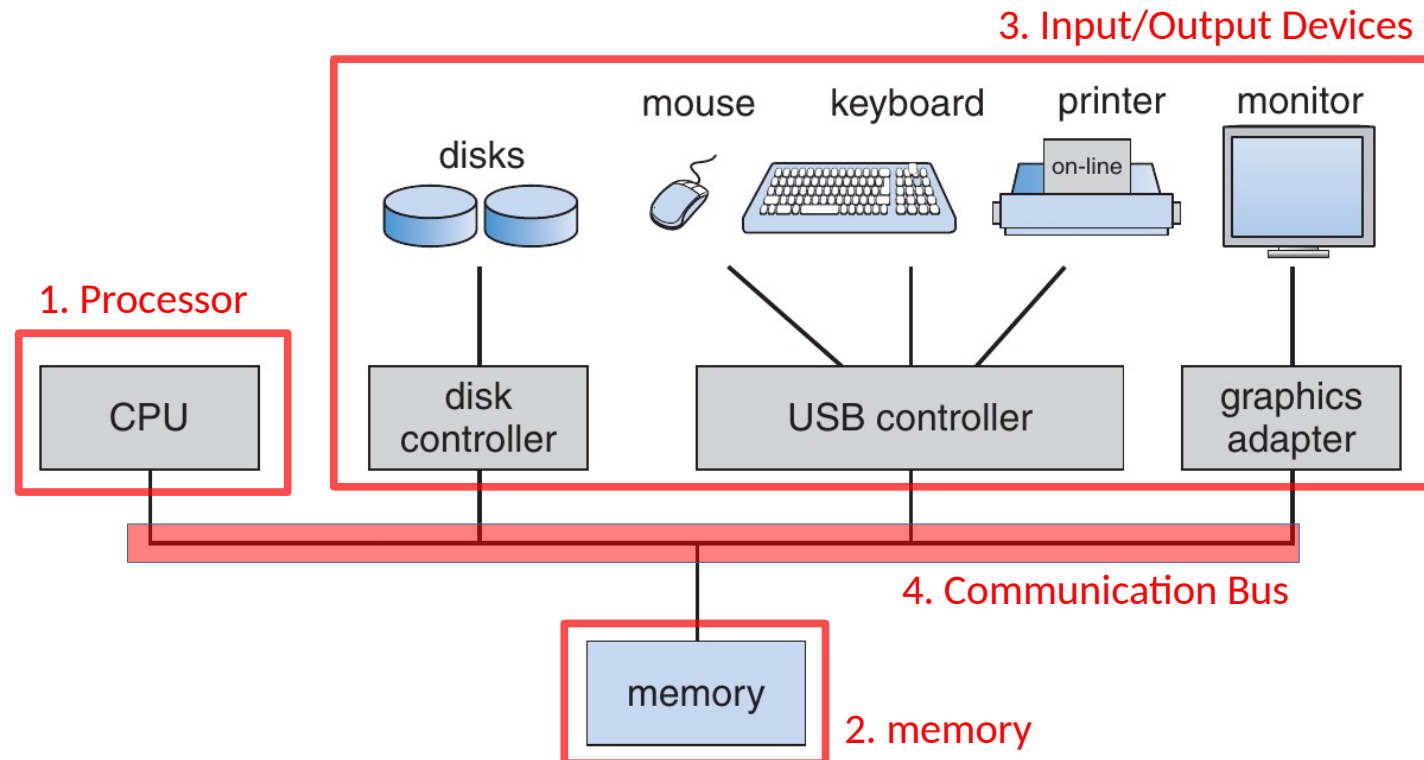
Memory Management

[Organizing Processes in Main Memory]

José Oramas

Introduction

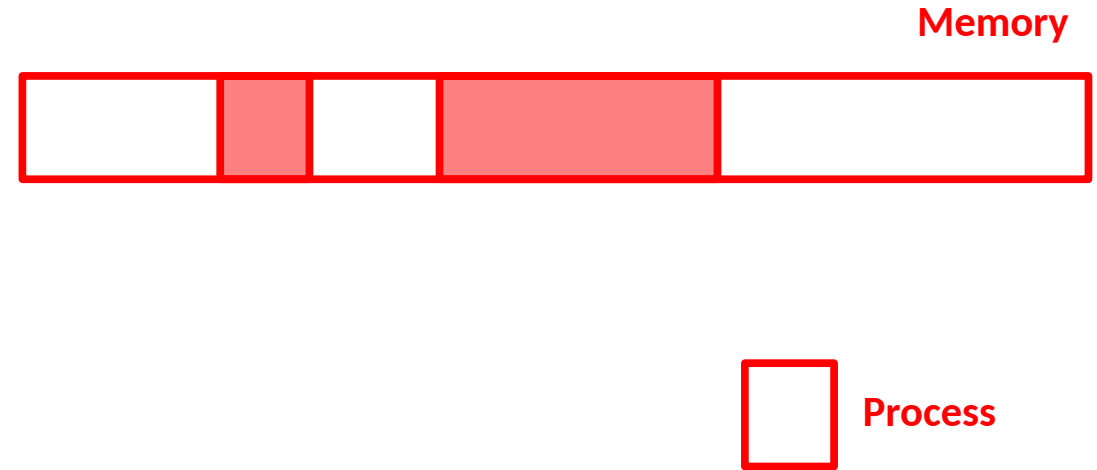
- Optimal use of the processor → multiple processes share memory
- The OS must determine: (1) which process are present in memory
(2) how they are organized



Memory Management

Address Translation

[handled by the loader, assisted by the linker]

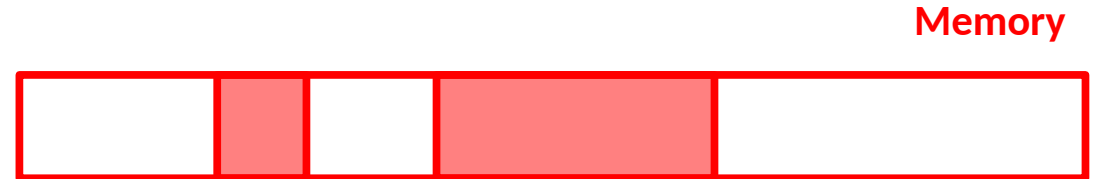


Memory Management

Address Translation

[handled by the loader, assisted by the linker]

- **At compile time**
→ absolute/physical address, location is rigid.



Memory Management

Address Translation

[handled by the loader, assisted by the linker]



- **At compile time**

→ absolute/physical address, location is rigid.



- **At load time**

→ relative/logical addresses mapped to absolute, once translated location is fixed

Memory Management

Address Translation

[handled by the loader, assisted by the linker]



- **At compile time**

→ absolute/physical address, location is rigid.



- **At load time**

→ relative/logical addresses mapped to absolute, once translated location is fixed

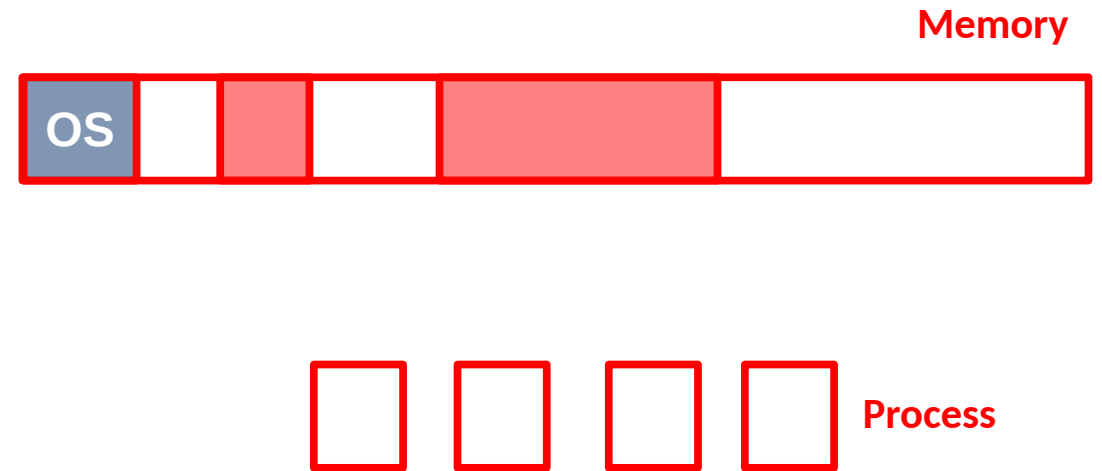
- **At run-time,**

→ relative/logical addresses, location is flexible

Memory Management

Memory Organization

- In most systems, part of the memory will be reserved for the OS.
- *This session:* focus on the management of memory to be shared by the different user processes.
- Three main alternatives:
 - Partitioning
 - Paging
 - Segmentation.



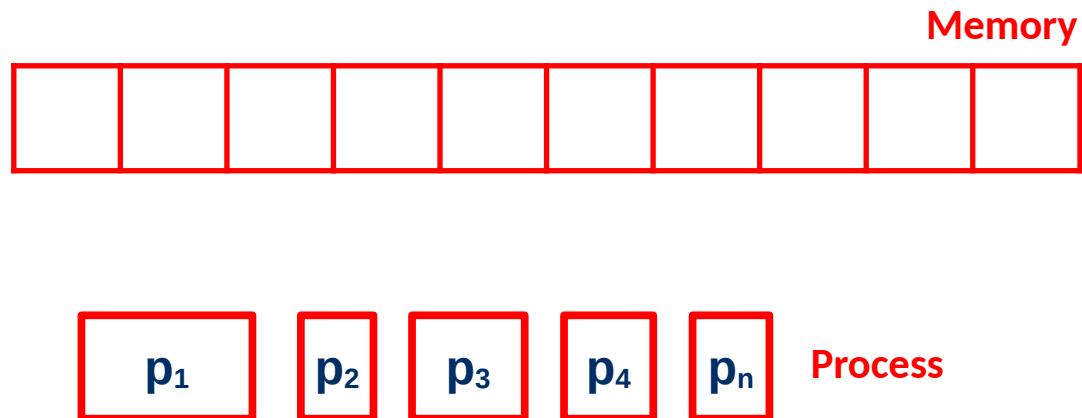
Partitioning

[dividing memory into pieces]

Partitioning

Fixed Partitioning – Fixed Size

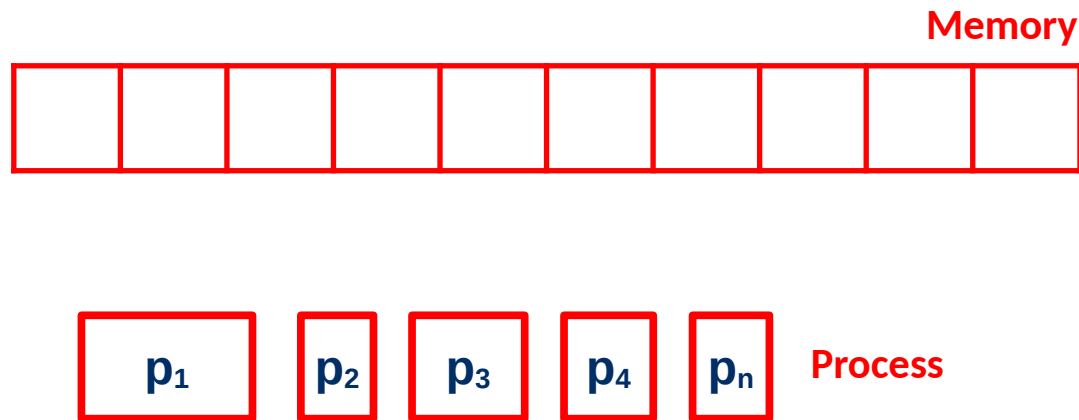
- Divide memory into partitions of *fixed* size
- Each process image gets a partition assigned



Partitioning

Fixed Partitioning – Fixed Size

- Divide memory into partitions of *fixed* size
- Each process image gets a partition assigned



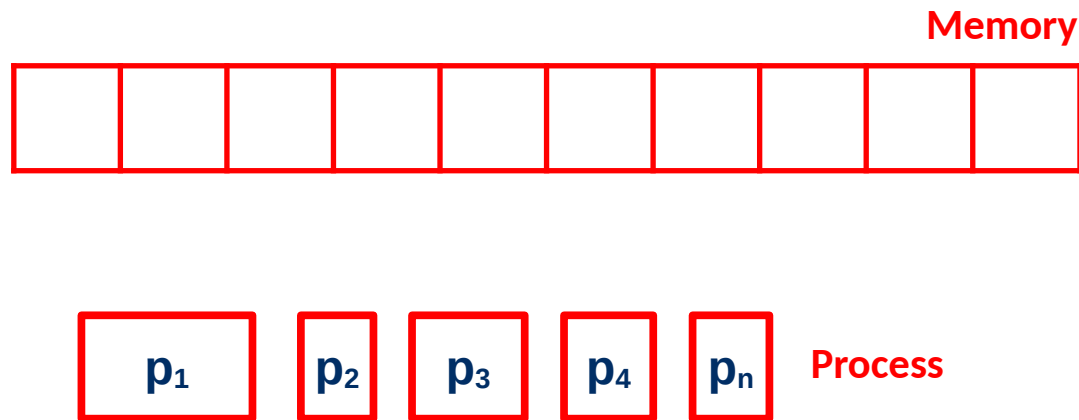
Any problem?

- Internal fragmentations occur
~half of the partition is unused
- Processes too large to fit
- Limitations
 - Max. size of process image
 - # of processes in memory

Partitioning

Fixed Partitioning – Fixed Size

- Divide memory into partitions of *fixed* size
- Each process image gets a partition assigned



Any problem?

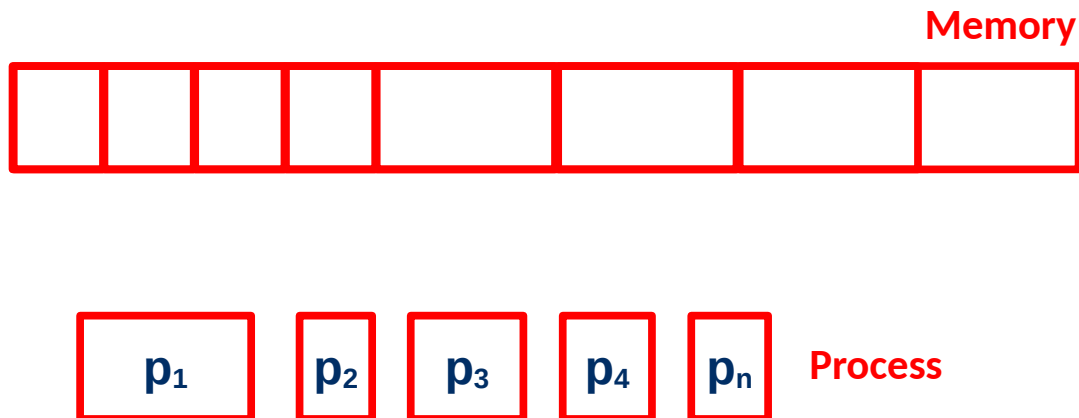
- Internal fragmentations occur
~half of the partition is unused
- Processes too large to fit
- Limitations
 - Max. size of process image
 - # of processes in memory

- **Solution:** have partitions of multiple sizes
→ how to choose/define such sizes?

Partitioning

Fixed Partitioning – Variable Size

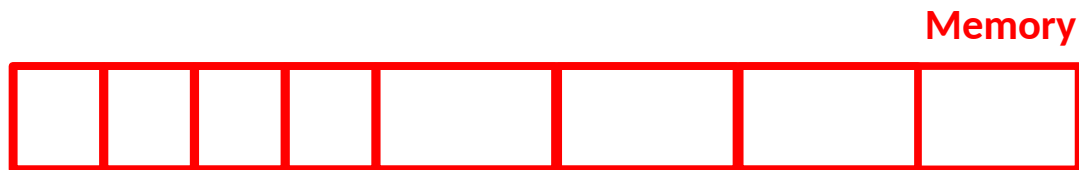
- Divide memory into partitions of *variable* size
- Allows for larger processes
- Less internal fragmentation losses



Partitioning

Fixed Partitioning – Variable Size

- Divide memory into partitions of *variable* size
- Allows for larger processes
- Less internal fragmentation losses



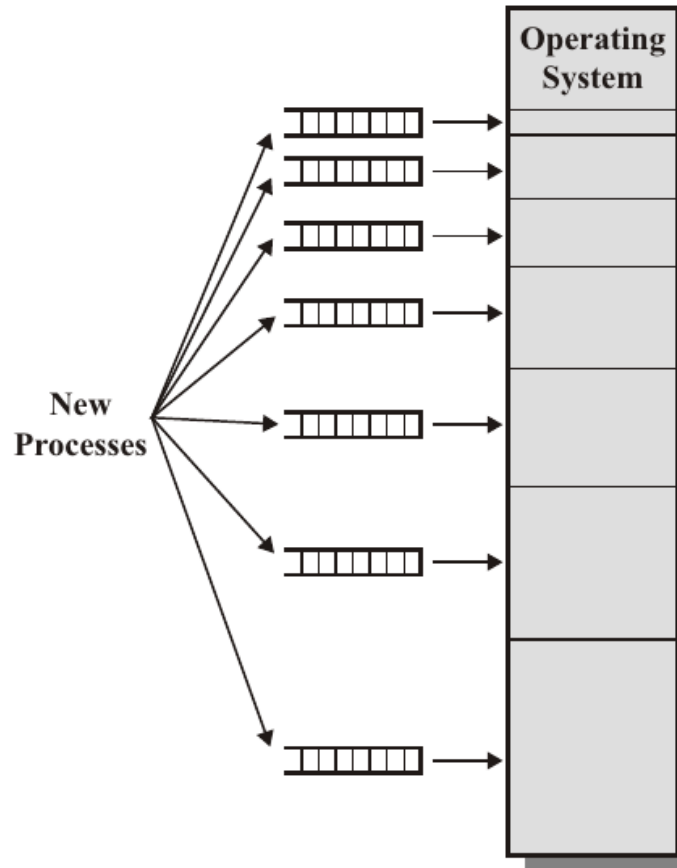
Any problem/challenge?

- How to decide where to add which process?
→ Process-placement algorithms needed

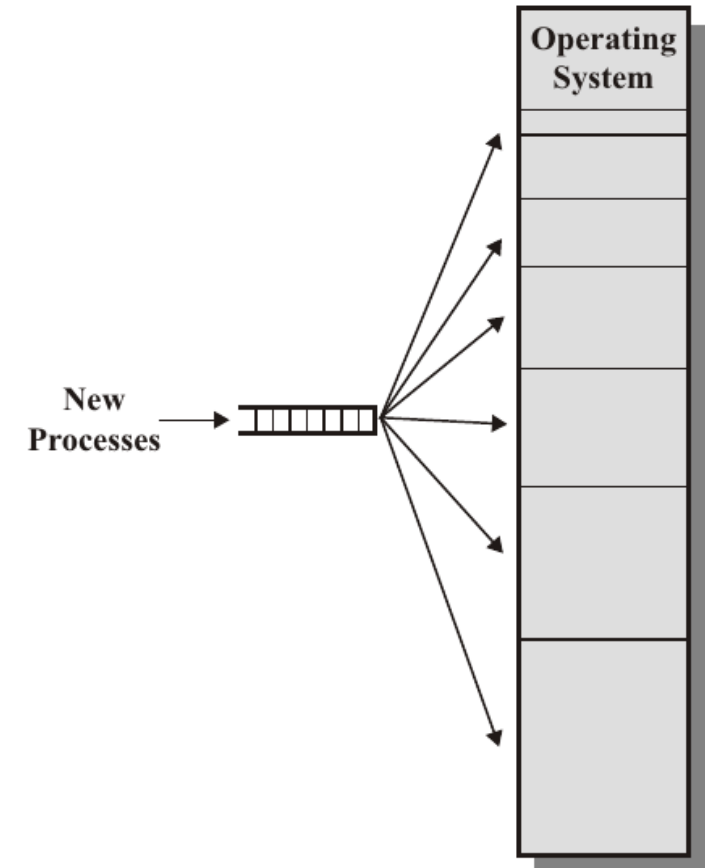
Partitioning

Fixed Partitioning – Variable Size – Placement Algorithm

- Wait until a partition is available
→ Maximizes memory usage
- Take a sufficiently large partition
→ Maximizes number of partitions in use



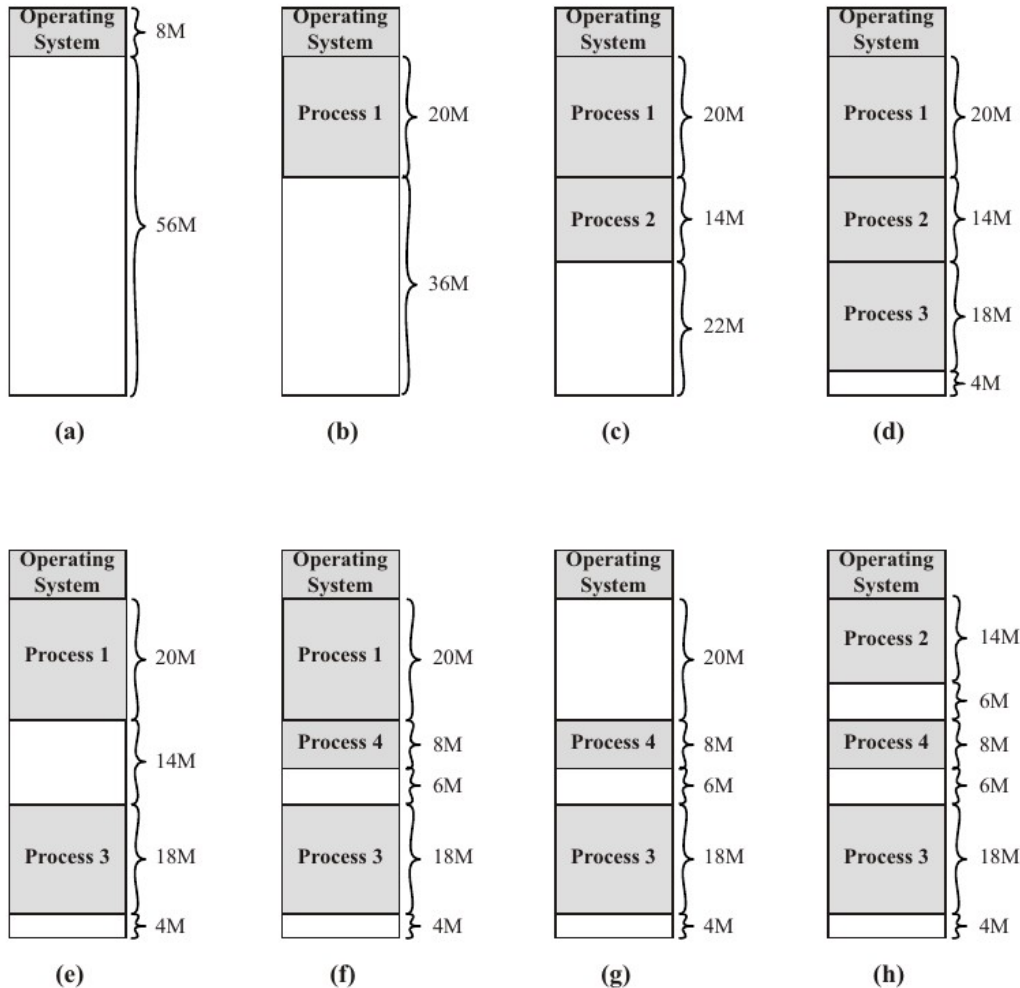
(a) One process queue per partition



(b) Single queue

Partitioning

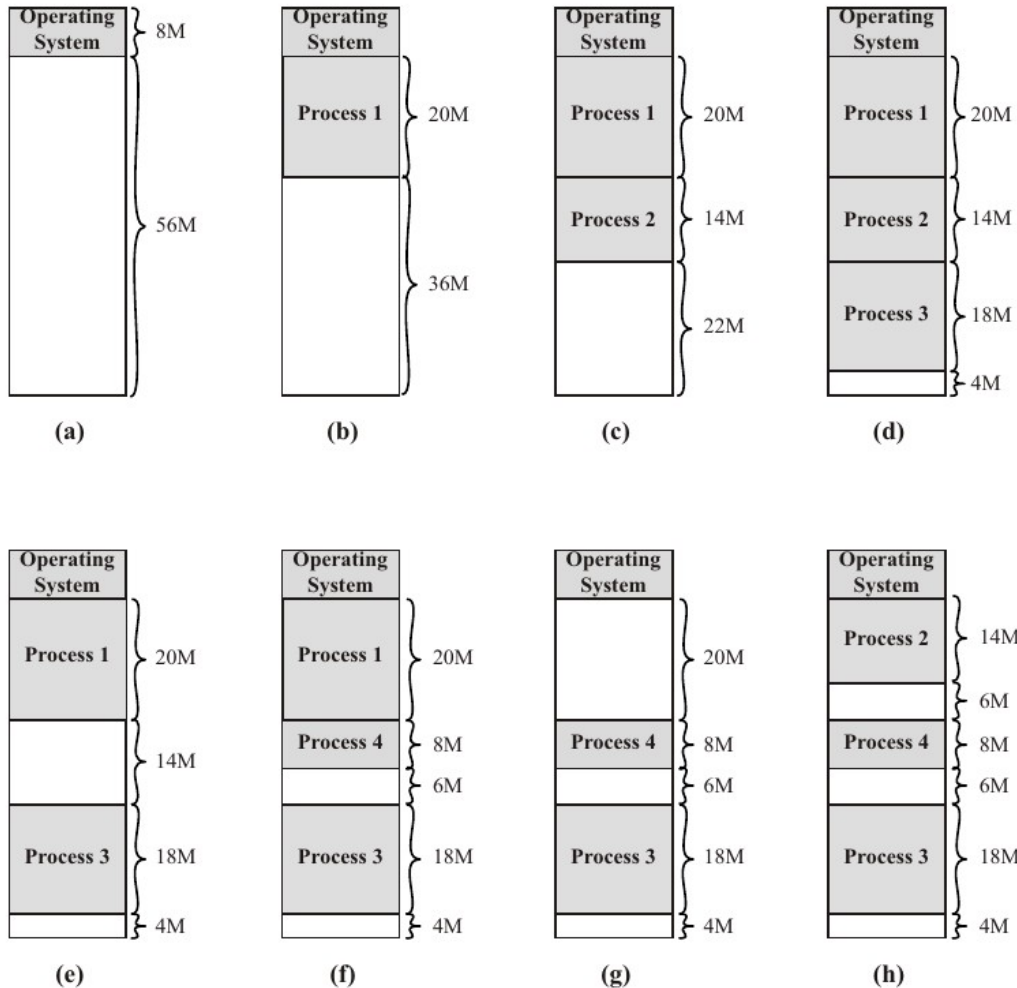
Dynamic Partitioning



- Create partitions dynamically to match the required size.
- New processes are added/removed all the time.

Partitioning

Dynamic Partitioning



- Create partitions dynamically to match the required size.
- New processes are added/removed all the time.

Any problem/challenge?

- Produces memory holes(voids) where processes cannot fit
→ external fragmentation
- When placing a new partition, ensure existing partitions get connected
→ no holes

Partitioning

Dynamic Partitioning – 50% rule

How many holes should we expect?

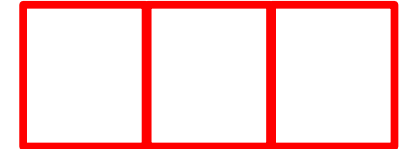
- H : number of holes, $E(H)$: average number of holes
- p_i : probability that process is surrounded by i holes ($i=0,1,2$)
 - Each time a process is added:
 - H constant (*good assumption?*)
 - - Each time a process (ends) is removed:
 - average value $H + p_0 - p_2$
- So there must be as many processes with 0 and 2 holes as neighbors
→ on average 1 hole as neighbor

Note: the neighbor of a process is neighbor of another process

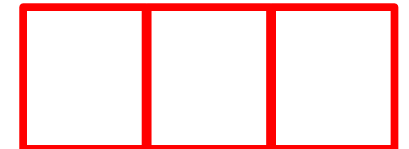
⇒ ***2x as many processes as holes***

Memory region

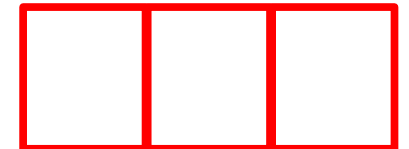
P_0



P_1



P_2



Partitioning

Dynamic Partitioning – 50% rule (A Closer Look)

How many holes should we expect?

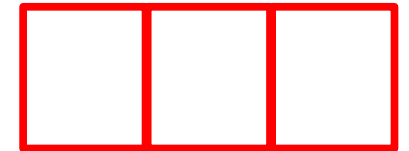
- H : number of holes, $E(H)$: average number of holes
- p_i : probability that process is surrounded by i holes ($i=0,1,2$)
 - Each time a process is added:
 - H constant (*good assumption?*)
 - - Each time a process (ends) is removed:
 - average value $H + p_0 - p_2$
- So there must be as many processes with 0 and 2 holes as neighbors
→ on average 1 hole as neighbor

Note: the neighbor of a process is neighbor of another process

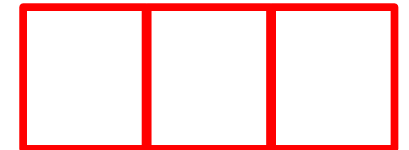
⇒ ***2x as many processes as holes***

Memory region

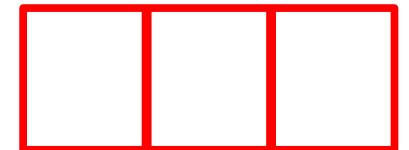
P_0



P_1



P_2



Partitioning

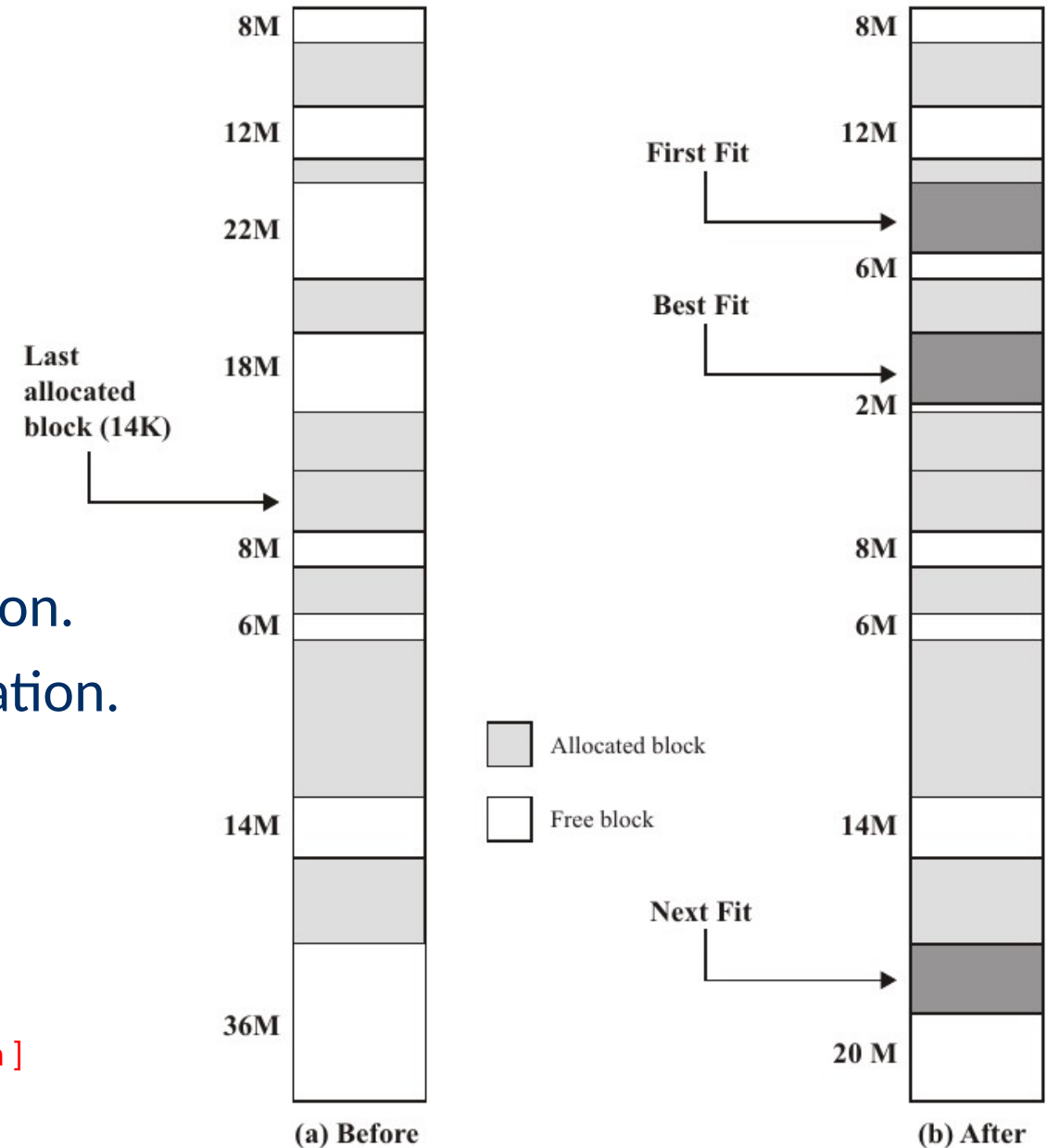
Dynamic Partitioning

– Placement Algorithms

- **Best-fit:** optimize size match
→ produces many little holes.
- **First-fit:** looks for the first sufficient location.
- **Next-fit:** the closest to the previous allocation.
- **Worst-fit:** largest hole
→ may block larger future processes.

[Please see lecture notes and videos for experimental validation]

t_0 : [Assuming a 16M process] t_1 :

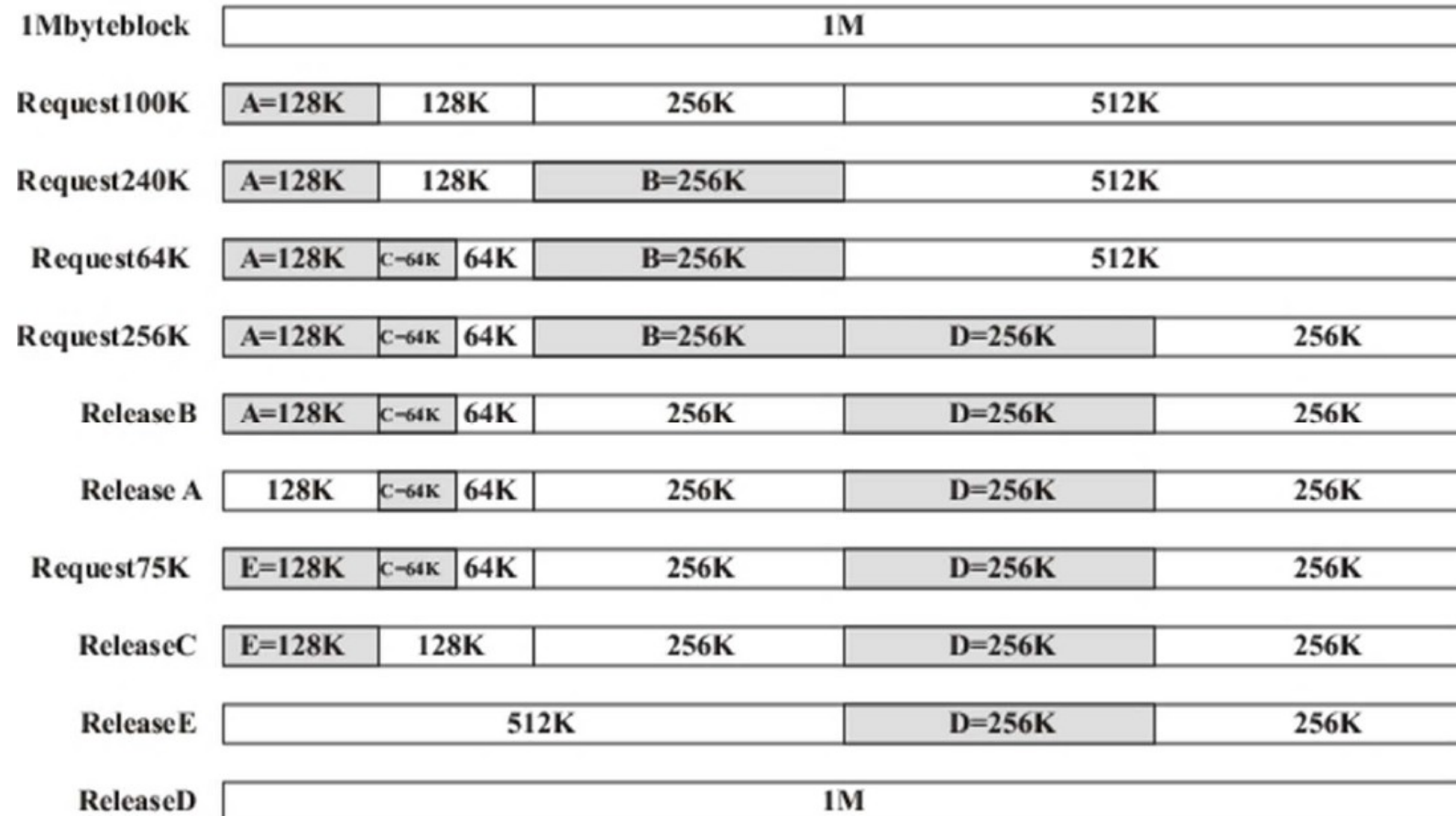


Partitioning

Buddy System

Consider:

- Memory size: 2^H and $2^L < \text{block size} < 2^H$
- If a block of the needed size (k) is not present:
 - Divide the block recursively
 - Stop when size k is reached.

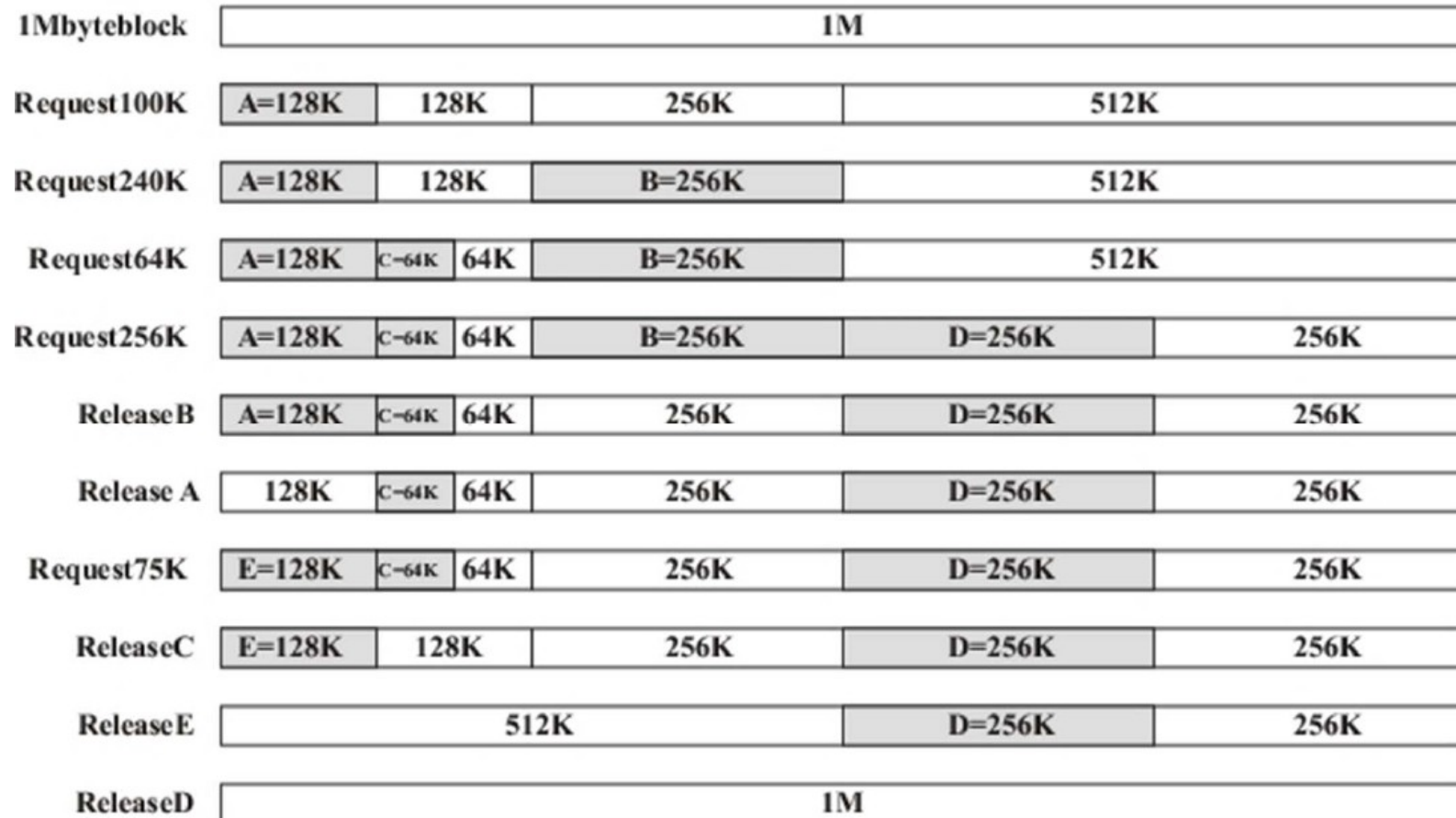


Partitioning

Buddy System

Consider:

- Memory size: 2^H and $2^L < \text{block size} < 2^H$
- If a block of the needed size (k) is not present:
 - Divide the block recursively
 - Stop when size k is reached.
- If a block is released (process is removed from memory):
 - The block may be rejoined with its (free) buddy to a larger block

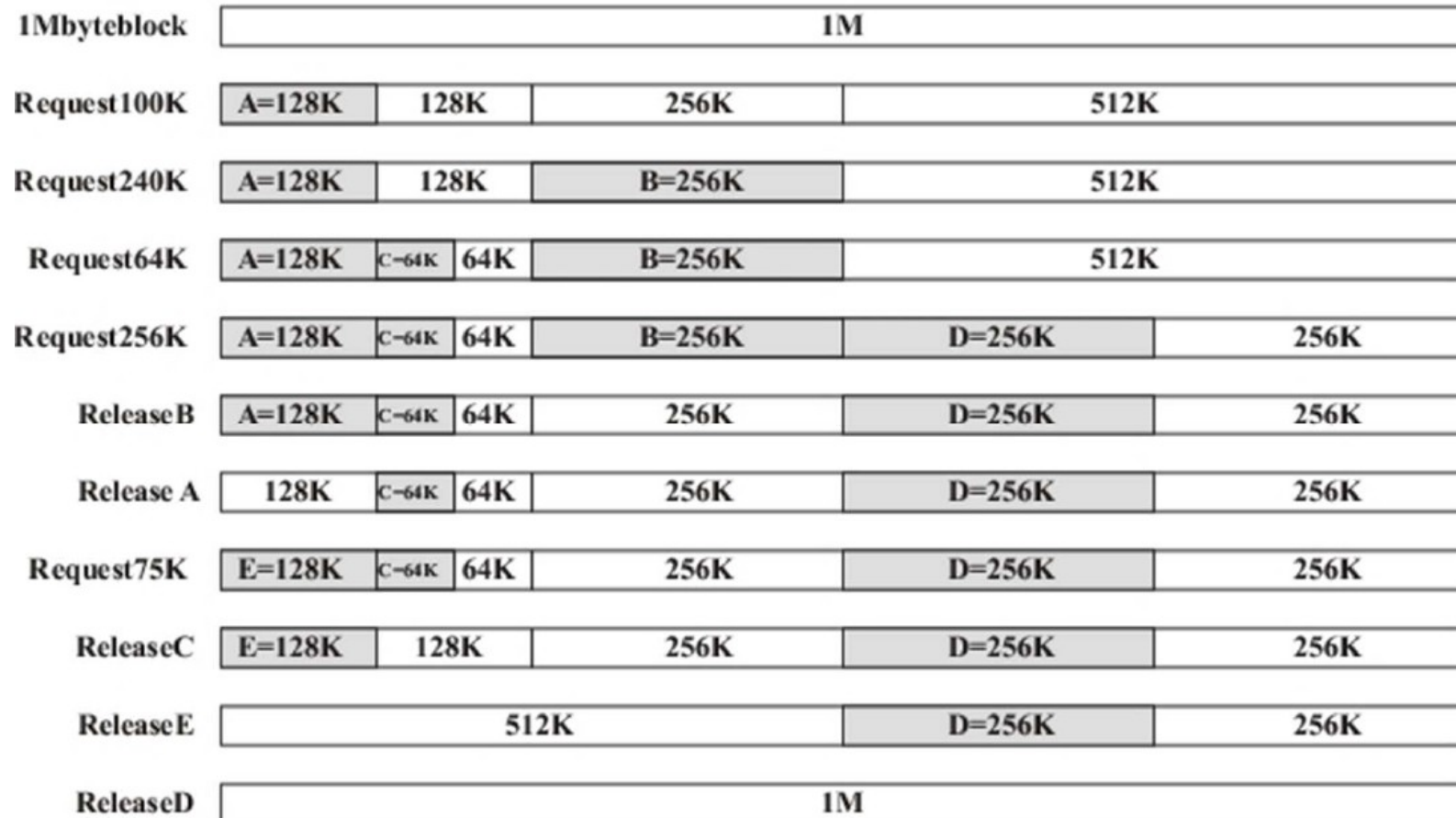


Partitioning

Buddy System

Consider:

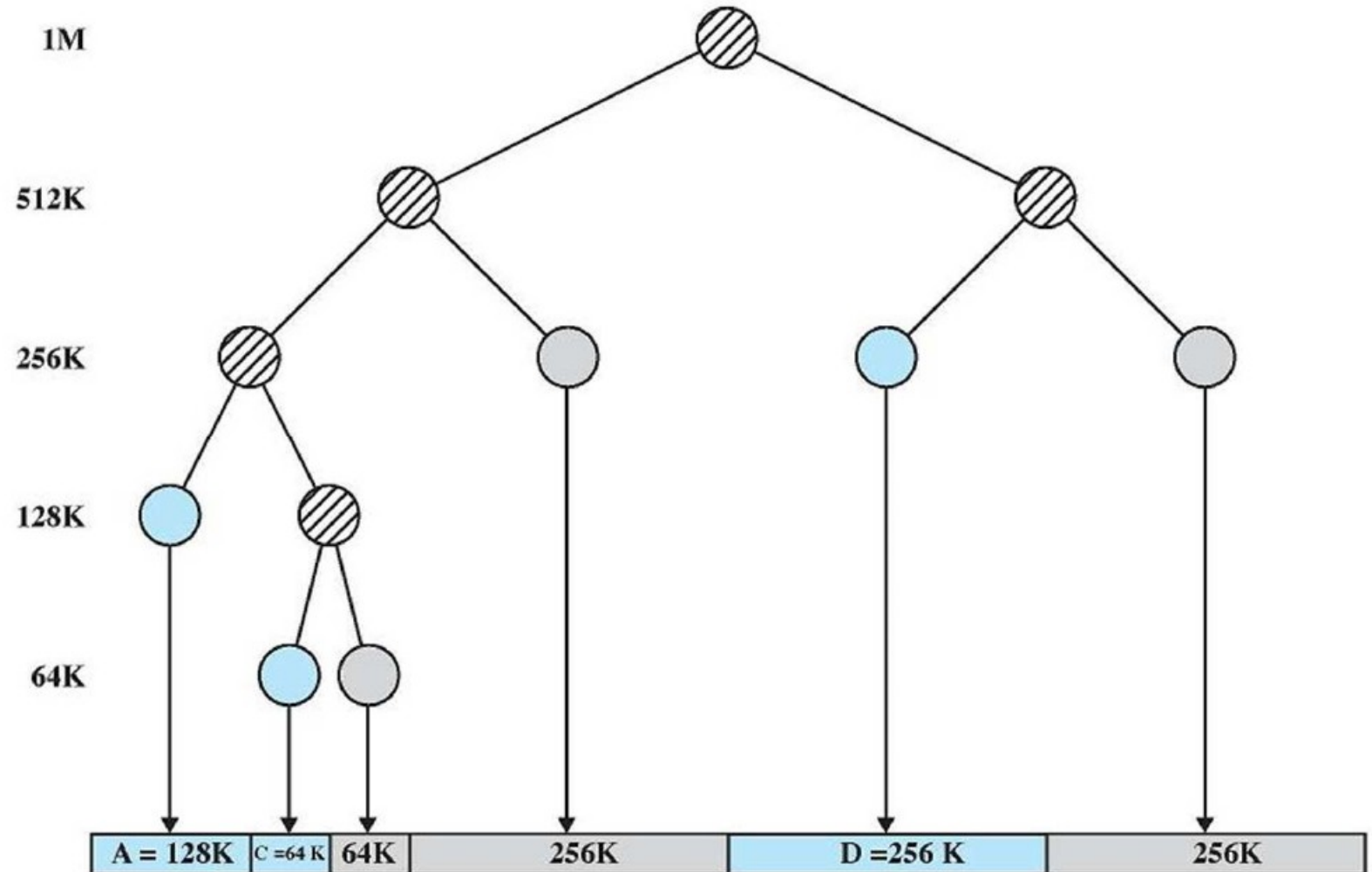
- Memory size: 2^H and $2^L < \text{block size} < 2^H$
- If a block of the needed size (k) is not present:
 - Divide the block recursively
 - Stop when size k is reached.
- If a block is released (process is removed from memory):
 - The block may be rejoined with its (free) buddy to a larger block
- 75% memory usage



Partitioning

Buddy System

- Implementation
 - List per blocksize
 - Tree



Paging

[dividing memory and processes into pieces]

Paging

Description

- Divide the entire memory into small blocks (*page frames*) of fixed size
- Divide the image of a process into pieces (*pages*) with the same size as frames.



- **Relative addresses:** page number + offset
- For the programmer/compiler the process image appears as a coherent block
→ “Transparent”

Paging

Description

- Divide the entire memory into small blocks (page frames) of fixed size
- Divide the image of a process into pieces (pages) with the same size as frames.



Memory



Process

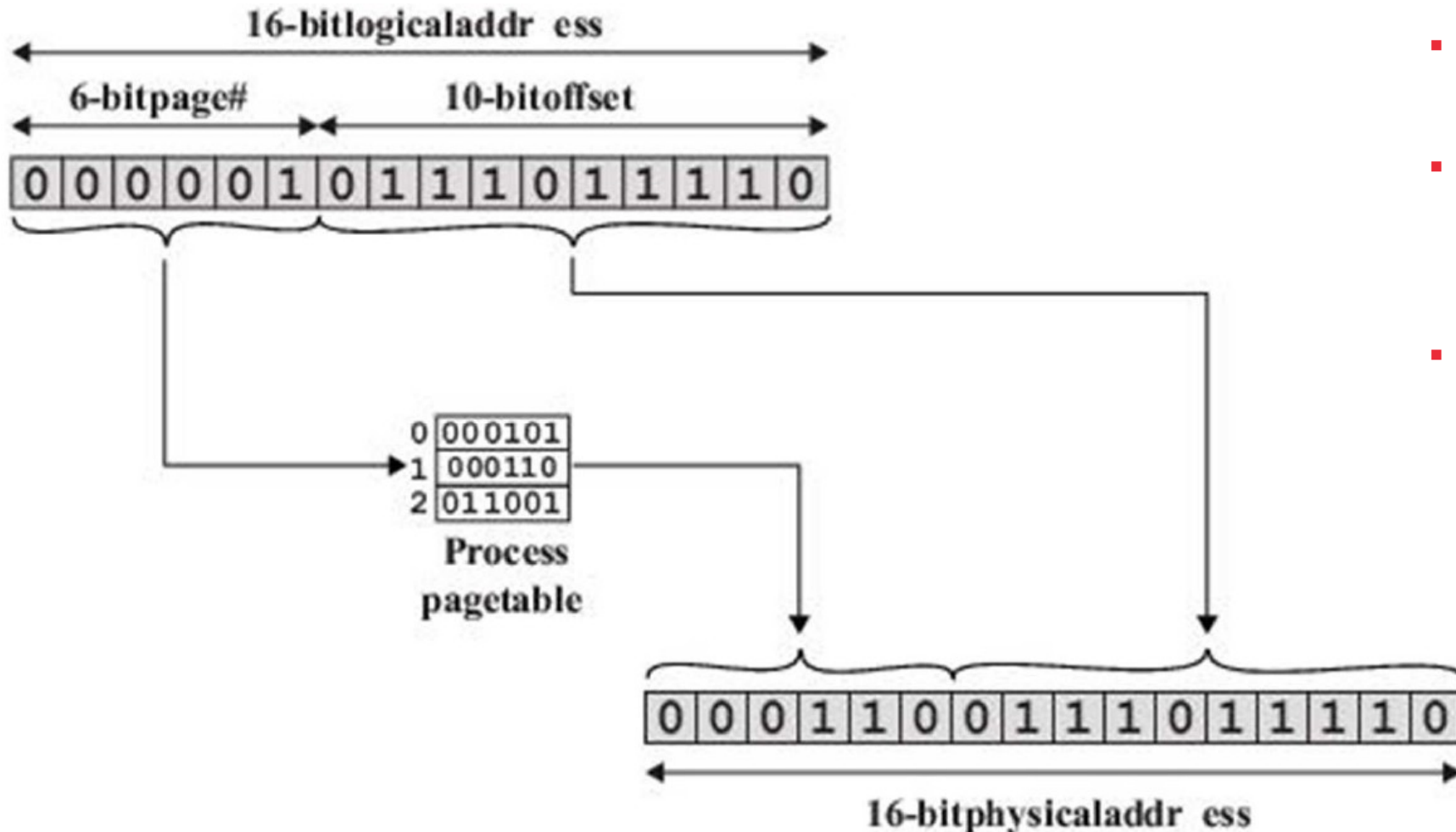
What about fragmentation?

- No external fragmentation
- Small internal fragmentation
→ size of process may not be divisible by frame size

- **Relative addresses:** page number + offset
- For the programmer/compiler the process image appears as a coherent block
→ “Transparent”

Paging

Memory Translation

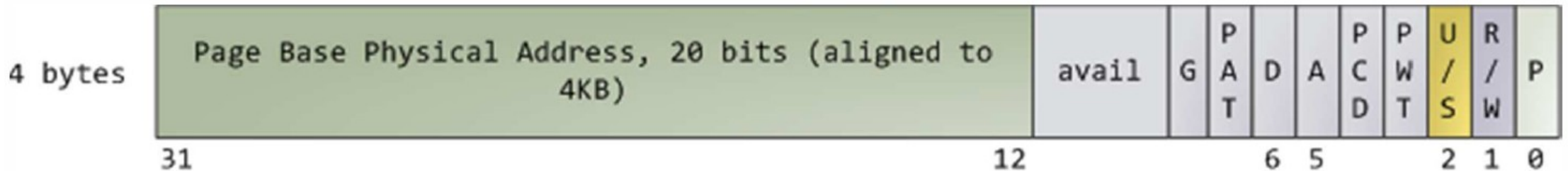


- Logical address: $n + m$ bits
[frame size = 2^m]
- Check process page table
[first n bits of physical address]
- Concatenate the remaining m bits from logical address

x86 Paging

x86 architecture (32-bit):

- 4096-byte frames, 20-bit page number
- 4-byte page table entry (12 bit for other purposes)



- **P bit:** page present in memory?
- **U/S bit:** page may be requested in user mode? (protect kernel memory)
- **R/W bit:** write to page allowed? (protect instructions)
- **D bit:** page is changed?
- **A bit:** page is used?

Segmentation

[like paging but with pieces of variable size]

Segmentation

Description

- Similar to paging but with partitions (*segments*) of variable size
- **Addresses:** segment number + offset value



Segmentation

Description

- Similar to paging but with partitions (segments) of variable size
- **Addresses:** segment number + offset value

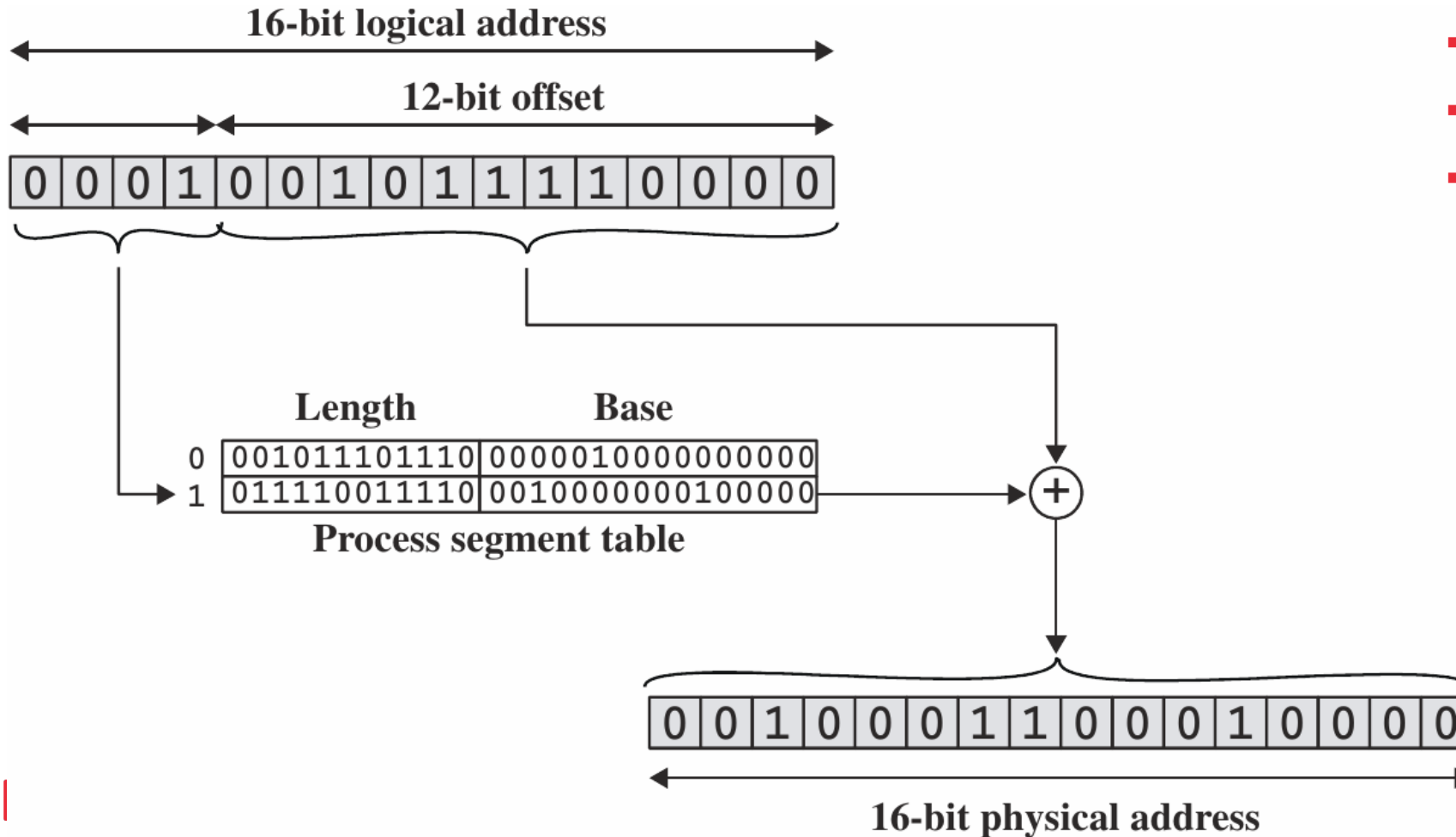


What about fragmentation?

- No Internal fragmentation
- External fragmentation may occur
[less than with dynamic partitioning]

Segmentation

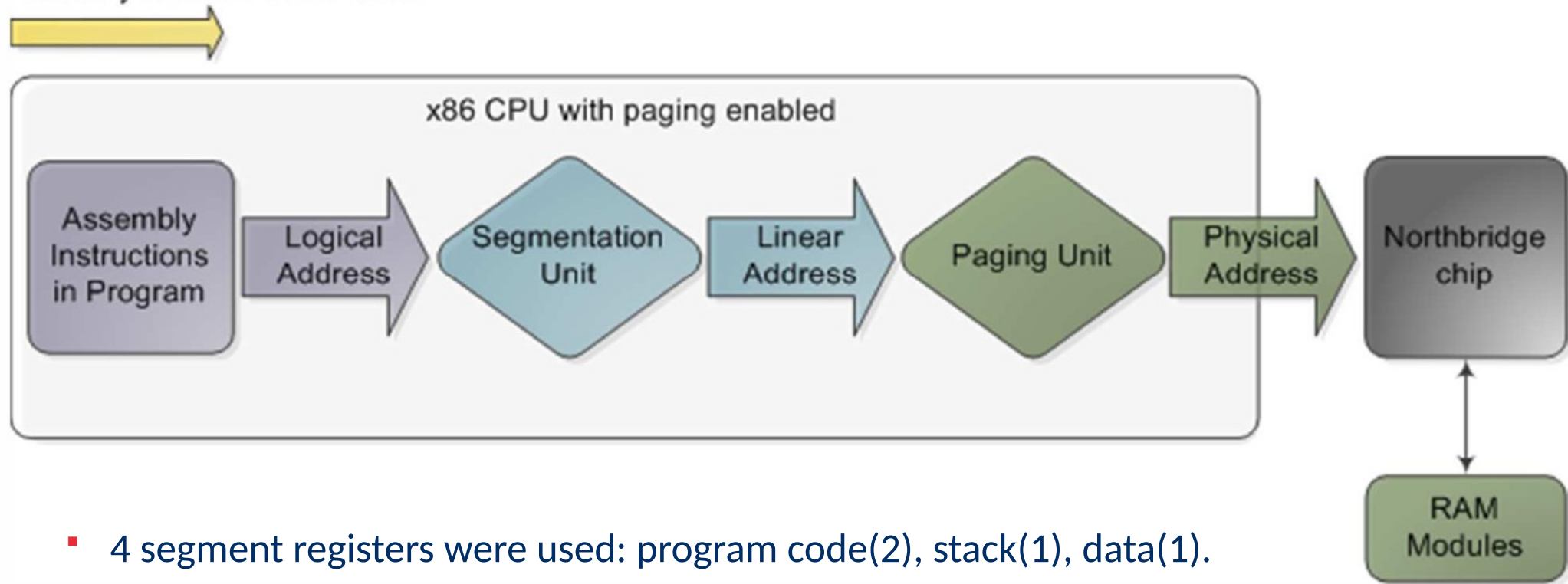
Memory Translation



- Logical address: $n + m$ bits
- Check process segment table
- If (offset < length)
Physical address = base value + offset

x86 Segmentation

Memory Translation



- 4 segment registers were used: program code(2), stack(1), data(1).
- Some of these registers access to the programmer → not “transparent”.
- 2 modes of segmentation: *real mode* and *protected mode*

Key points

[Finally :D]

Memory Management

Pay attention to...

- Advantages and disadvantages of fixed and dynamic partitioning
- Placement algorithms
- How does the buddy system work?
- Paging and Segmentation and their relationship to fixed and dynamic partitioning

Further Reading

Essential: Cache Memory

- OS Course Notes - Chapter 3.
- Blackboard – Recorded lecture from Prof. Van Houdt.

Questions?



Operating Systems

[1500WETOPS]

José Oramas